

Lab_11

Hadia Beshir

5. The EBI AlphaFold database

Querying the AlphaFold database

1. Sequence to search AFDB: PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEM-SLPGRWKPKMIGGIGGFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRN-LLTQIGCTLNF
2. Examine one of your top hits

Protein Name: Gag-Pol polyprotein UniProt ID: P04585 Organism: Human immunodeficiency virus type 1 (HIV-1)

3. Visit the corresponding AFDB structure page. It shows basic information about the protein (drawn from UniProt), and three separate outputs from AlphaFold.

My model is a dark blue color. This shows that my model is very confident (pLDDT > 90). It also states that the average pLDDT is 96.78 which is considered very high.

After locating the C-terminus of the model in this case was the last letter (F). It had a pLDDT value of 88.81 (Confident). It is still considered high, just not as high as the average. it was a light/teal blue color.

The PAE matrix is all dark green. The C-terminus is highly reliable.

6. Generating your own structure positions
7. Interpreting Results

The graph generated is dark orange/red. This shows that the structure has high confidence.
figure 18.

```
library(imager)
```

Loading required package: magrittr

Attaching package: 'imager'

The following object is masked from 'package:magrittr':

add

The following objects are masked from 'package:stats':

convolve, spectrum

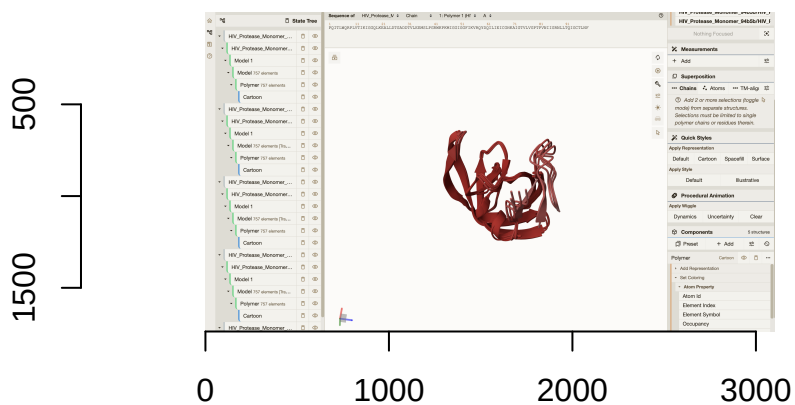
The following object is masked from 'package:graphics':

frame

The following object is masked from 'package:base':

save.image

```
img <- load.image("Figure 18..png")
plot(img)
```



8. Custom Analysis

```
results_dir <- "HIV_Protease_Dimer_23119"
pdb_files <- list.files(path=results_dir,
                        pattern="*.pdb",
                        full.names = TRUE)
```

```
# Print our PDB file names
basename(pdb_files)
```

```
[1] "HIV_Protease_Dimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_model_4_seed_000.pdb"
[2] "HIV_Protease_Dimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_model_1_seed_000.pdb"
[3] "HIV_Protease_Dimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_model_5_seed_000.pdb"
[4] "HIV_Protease_Dimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_model_2_seed_000.pdb"
[5] "HIV_Protease_Dimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_model_3_seed_000.pdb"
```

```
library(bio3d)
```

```
pdbbs <- pdbaln(pdb_files, fit=TRUE, exefile="msa")
```

Reading PDB files:

```
HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_001_alphafold2_multimer_v3_r
HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_002_alphafold2_multimer_v3_r
HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_003_alphafold2_multimer_v3_r
HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_004_alphafold2_multimer_v3_r
HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_005_alphafold2_multimer_v3_r
.....
```

Extracting sequences

```
pdb/seq: 1    name: HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_001_alpha
pdb/seq: 2    name: HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_002_alpha
pdb/seq: 3    name: HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_003_alpha
pdb/seq: 4    name: HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_004_alpha
pdb/seq: 5    name: HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119_unrelaxed_rank_005_alpha
```

```
pdbbs
```

```
[Truncated_Name:1]HIV_Protea      1      .      .      .      .      50
PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
```

```

[Truncated_Name:2]HIV_Protea PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:3]HIV_Protea PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:4]HIV_Protea PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
[Truncated_Name:5]HIV_Protea PQITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGI
*****
1 . . . . 50

51 . . . . 100
[Truncated_Name:1]HIV_Protea GGFIVKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:2]HIV_Protea GGFIVKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:3]HIV_Protea GGFIVKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:4]HIV_Protea GGFIVKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
[Truncated_Name:5]HIV_Protea GGFIVKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNFP
*****
51 . . . . 100

101 . . . . 150
[Truncated_Name:1]HIV_Protea QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:2]HIV_Protea QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:3]HIV_Protea QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:4]HIV_Protea QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
[Truncated_Name:5]HIV_Protea QITLWQRPLVTIKIGGQLKEALLDTGADDTVLEEMSLPGRWKPKMIGGIG
*****
101 . . . . 150

151 . . . . 198
[Truncated_Name:1]HIV_Protea GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:2]HIV_Protea GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:3]HIV_Protea GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:4]HIV_Protea GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
[Truncated_Name:5]HIV_Protea GFIKVRQYDQILIEICGHKAIGTVLVGPTPVNIIGRNLLTQIGCTLNF
*****
151 . . . . 198

```

Call:

```
pdbaln(files = pdb_files, fit = TRUE, exefile = "msa")
```

Class:

```
pdb, fasta
```

Alignment dimensions:

```
5 sequence rows; 198 position columns (198 non-gap, 0 gap)
```

```
+ attr: xyz, resno, b, chain, id, ali, resid, sse, call
```

RMSD is a standard measure of structural distance between coordinate sets. We can use the `rmsd()` function to calculate the RMSD between all pairs models

```
rd <- rmsd(pdbbs, fit=T)
```

Warning in `rmsd(pdbbs, fit = T)`: No indices provided, using the 198 non NA positions

```
range(rd)
```

```
[1] 0.000 14.754
```

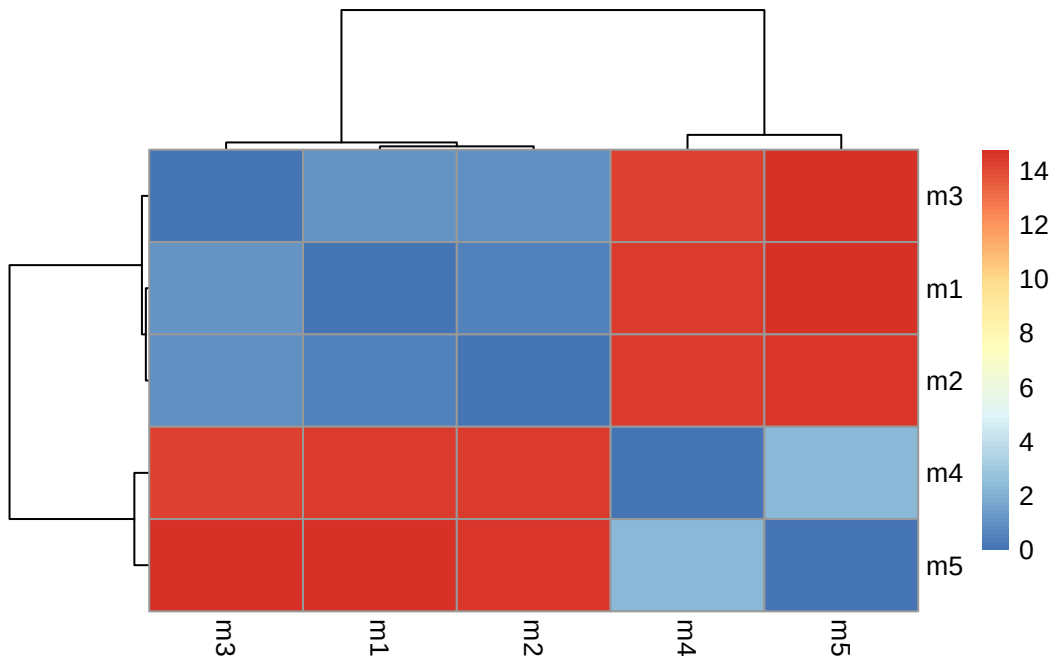
Draw a heatmap of these RMSD matrix values

```
library(pheatmap)
```

```
colnames(rd) <- paste0("m",1:5)
```

```
rownames(rd) <- paste0("m",1:5)
```

```
pheatmap(rd)
```

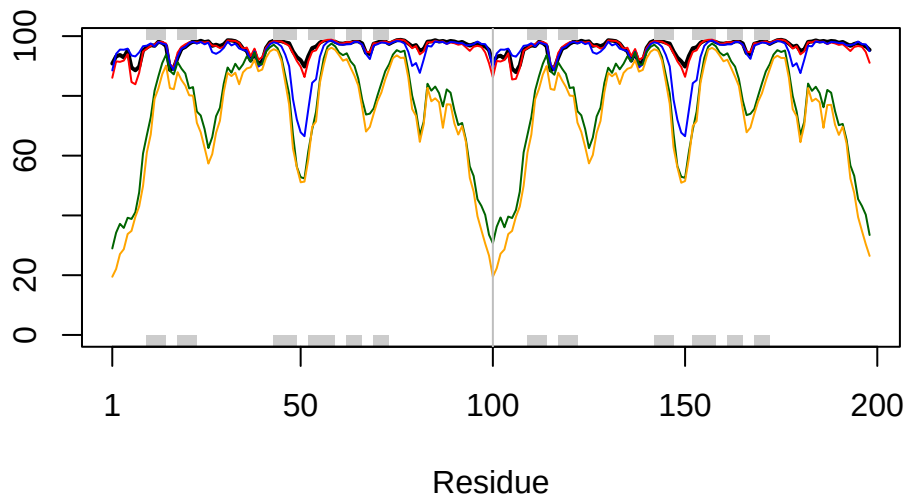


Now let's plot the pLDDT values across all models. Recall that this information is in the B-factor column of each model and that this is stored in our aligned pdbs object as `pdbs$b` with a row per structure/model.

```
pdb <- read.pdb("1hsg")
```

Note: Accessing on-line PDB file

```
plotb3(pdbs$b[1,], typ="l", lwd=2, sse=pdb)
points(pdbs$b[2,], typ="l", col="red")
points(pdbs$b[3,], typ="l", col="blue")
points(pdbs$b[4,], typ="l", col="darkgreen")
points(pdbs$b[5,], typ="l", col="orange")
abline(v=100, col="gray")
```



We can improve the superposition/fitting of our models by finding the most consistent “rigid core” common across all the models. For this we will use the `core.find()` function:

```
core <- core.find(pdbs)
```

core size 197 of 198 vol = 9885.854

core size 196 of 198	vol = 6898.765
core size 195 of 198	vol = 1338.085
core size 194 of 198	vol = 1040.68
core size 193 of 198	vol = 951.864
core size 192 of 198	vol = 899.09
core size 191 of 198	vol = 834.739
core size 190 of 198	vol = 771.346
core size 189 of 198	vol = 733.072
core size 188 of 198	vol = 697.287
core size 187 of 198	vol = 659.749
core size 186 of 198	vol = 625.281
core size 185 of 198	vol = 589.548
core size 184 of 198	vol = 568.261
core size 183 of 198	vol = 545.023
core size 182 of 198	vol = 512.897
core size 181 of 198	vol = 490.731
core size 180 of 198	vol = 470.274
core size 179 of 198	vol = 450.739
core size 178 of 198	vol = 434.743
core size 177 of 198	vol = 420.345
core size 176 of 198	vol = 406.667
core size 175 of 198	vol = 393.342
core size 174 of 198	vol = 382.403
core size 173 of 198	vol = 372.867
core size 172 of 198	vol = 357.002
core size 171 of 198	vol = 346.576
core size 170 of 198	vol = 337.454
core size 169 of 198	vol = 326.667
core size 168 of 198	vol = 314.958
core size 167 of 198	vol = 304.136
core size 166 of 198	vol = 294.561
core size 165 of 198	vol = 285.657
core size 164 of 198	vol = 278.893
core size 163 of 198	vol = 266.774
core size 162 of 198	vol = 259.003
core size 161 of 198	vol = 247.731
core size 160 of 198	vol = 239.849
core size 159 of 198	vol = 234.972
core size 158 of 198	vol = 230.071
core size 157 of 198	vol = 221.994
core size 156 of 198	vol = 215.629
core size 155 of 198	vol = 206.802
core size 154 of 198	vol = 196.993

core size 153 of 198	vol = 188.548
core size 152 of 198	vol = 182.271
core size 151 of 198	vol = 176.962
core size 150 of 198	vol = 170.72
core size 149 of 198	vol = 166.128
core size 148 of 198	vol = 159.805
core size 147 of 198	vol = 153.775
core size 146 of 198	vol = 149.101
core size 145 of 198	vol = 143.666
core size 144 of 198	vol = 137.146
core size 143 of 198	vol = 132.525
core size 142 of 198	vol = 127.239
core size 141 of 198	vol = 121.581
core size 140 of 198	vol = 116.782
core size 139 of 198	vol = 112.577
core size 138 of 198	vol = 108.177
core size 137 of 198	vol = 105.14
core size 136 of 198	vol = 101.256
core size 135 of 198	vol = 97.381
core size 134 of 198	vol = 92.98
core size 133 of 198	vol = 88.19
core size 132 of 198	vol = 84.035
core size 131 of 198	vol = 81.904
core size 130 of 198	vol = 78.024
core size 129 of 198	vol = 75.278
core size 128 of 198	vol = 73.058
core size 127 of 198	vol = 70.701
core size 126 of 198	vol = 68.979
core size 125 of 198	vol = 66.709
core size 124 of 198	vol = 64.378
core size 123 of 198	vol = 61.147
core size 122 of 198	vol = 59.031
core size 121 of 198	vol = 56.627
core size 120 of 198	vol = 54.016
core size 119 of 198	vol = 51.806
core size 118 of 198	vol = 49.652
core size 117 of 198	vol = 48.193
core size 116 of 198	vol = 46.648
core size 115 of 198	vol = 44.752
core size 114 of 198	vol = 43.292
core size 113 of 198	vol = 41.093
core size 112 of 198	vol = 39.147
core size 111 of 198	vol = 36.472


```

core size 110 of 198  vol = 34.117
core size 109 of 198  vol = 31.47
core size 108 of 198  vol = 29.448
core size 107 of 198  vol = 27.325
core size 106 of 198  vol = 25.822
core size 105 of 198  vol = 24.15
core size 104 of 198  vol = 22.648
core size 103 of 198  vol = 21.069
core size 102 of 198  vol = 19.953
core size 101 of 198  vol = 18.3
core size 100 of 198  vol = 15.723
core size 99 of 198   vol = 14.841
core size 98 of 198   vol = 11.646
core size 97 of 198   vol = 9.434
core size 96 of 198   vol = 7.354
core size 95 of 198   vol = 6.179
core size 94 of 198   vol = 5.666
core size 93 of 198   vol = 4.705
core size 92 of 198   vol = 3.665
core size 91 of 198   vol = 2.77
core size 90 of 198   vol = 2.151
core size 89 of 198   vol = 1.715
core size 88 of 198   vol = 1.15
core size 87 of 198   vol = 0.874
core size 86 of 198   vol = 0.685
core size 85 of 198   vol = 0.528
core size 84 of 198   vol = 0.37
FINISHED: Min vol ( 0.5 ) reached

```

```
core.inds <- print(core, vol=0.5)
```

```

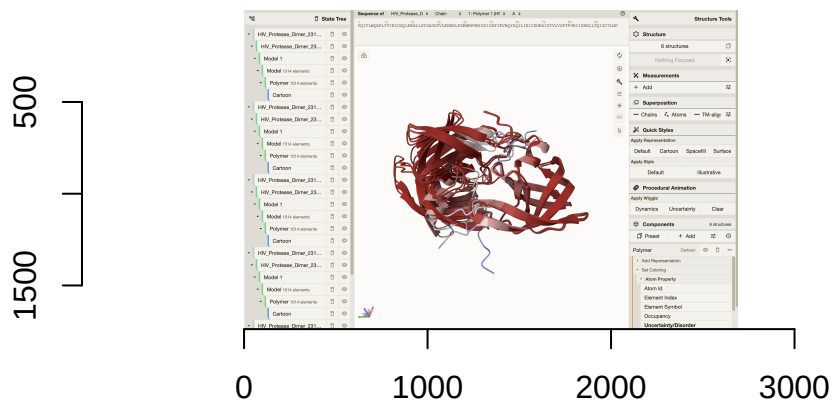
# 85 positions (cumulative volume <= 0.5 Angstrom^3)
  start end length
1     9  49     41
2    52  95     44

```

```
xyz <- pdbfit(pdb, core.inds, outpath="corefit_structures")
```

Figure 19.

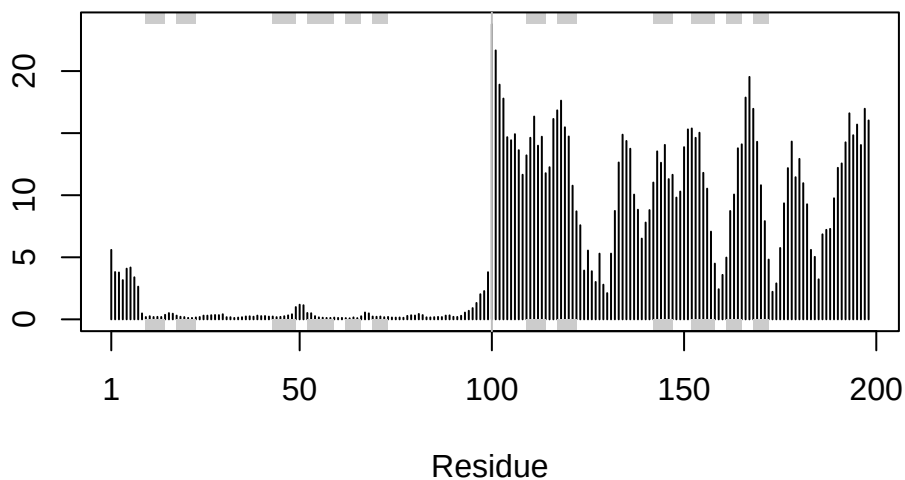
```
library(imager)
img2 <- load.image("Figure 19..png")
plot(img2)
```



Now we can examine the RMSF between positions of the structure. RMSF is an often used measure of conformational variance along the structure:

```
rf <- rmsf(xyz)

plotb3(rf, sse=pdb)
abline(v=100, col="gray", ylab="RMSF")
```



Predicted Alignment Error for domains

```
library(jsonlite)

# Listing of all PAE JSON files
pae_files <- list.files(path=results_dir,
                        pattern=".*model.*\\.json",
                        full.names = TRUE)
pae1 <- read_json(pae_files[1],simplifyVector = TRUE)
pae5 <- read_json(pae_files[5],simplifyVector = TRUE)

attributes(pae1)
```

```
$names
[1] "plddt" "max_pae" "pae" "ptm" "iptm"
```

```
# Per-residue pLDDT scores
# same as B-factor of PDB..
head(pae1$plddt)
```

```
[1] 90.81 93.25 93.69 92.88 95.25 89.44
```

The maximum PAE values are useful for ranking models. Here we can see that model 5 is much worse than model 1. The lower the PAE score the better. How about the other models, what are their max PAE scores?

```
pae1$max_pae
```

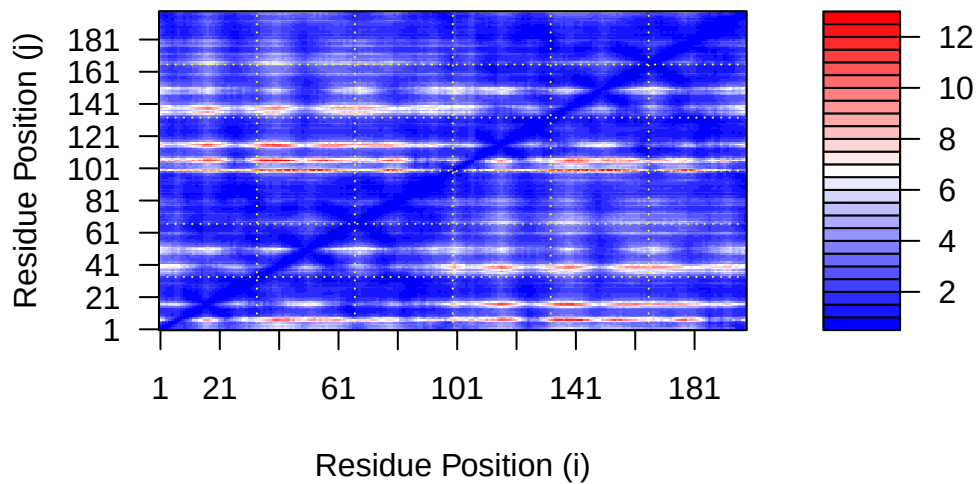
```
[1] 12.84375
```

```
pae5$max_pae
```

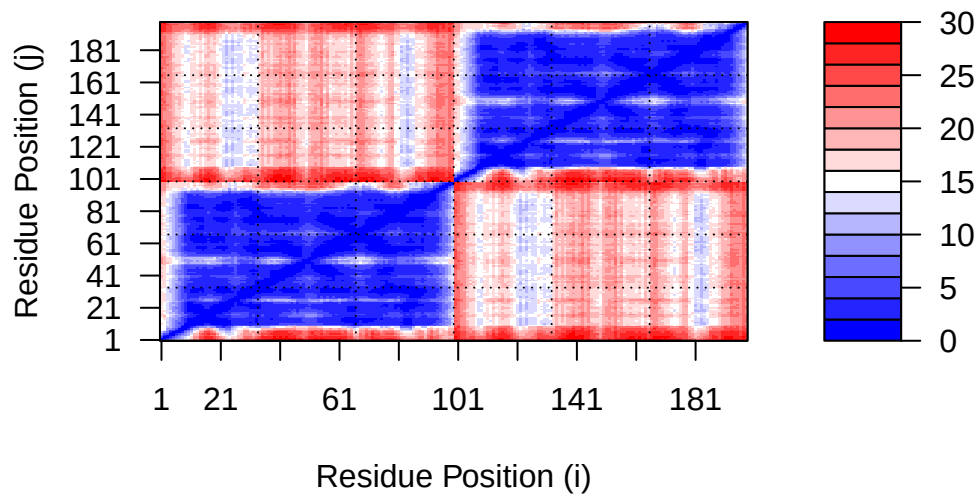
```
[1] 29.59375
```

We can plot the N by N (where N is the number of residues) PAE scores with ggplot or with functions from the Bio3D package:

```
plot.dmat(pae1$pae,  
          xlab="Residue Position (i)",  
          ylab="Residue Position (j)")
```

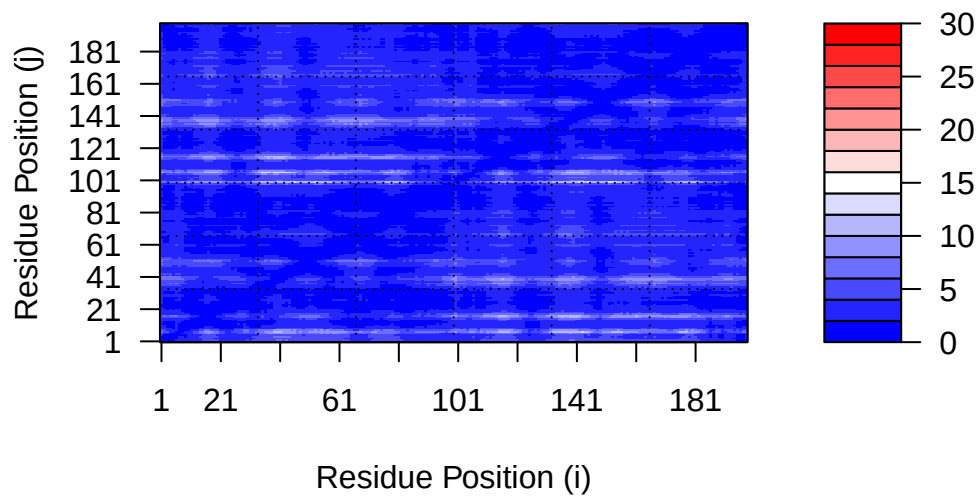


```
plot.dmat(pae5$paes,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



Same Z-range:

```
plot.dmat(pae1$paes,
          xlab="Residue Position (i)",
          ylab="Residue Position (j)",
          grid.col = "black",
          zlim=c(0,30))
```



Residue conservation from alignment file

```
aln_file <- list.files(path=results_dir,
                      pattern=".a3m$",
                      full.names = TRUE)
aln_file
```

```
[1] "HIV_Protease_Dimer_23119/HIV_Protease_Dimer_23119.a3m"
```

```
aln <- read.fasta(aln_file[1], to.upper = TRUE)
```

```
[1] " ** Duplicated sequence id's: 101 **"
[2] " ** Duplicated sequence id's: 101 **"
```

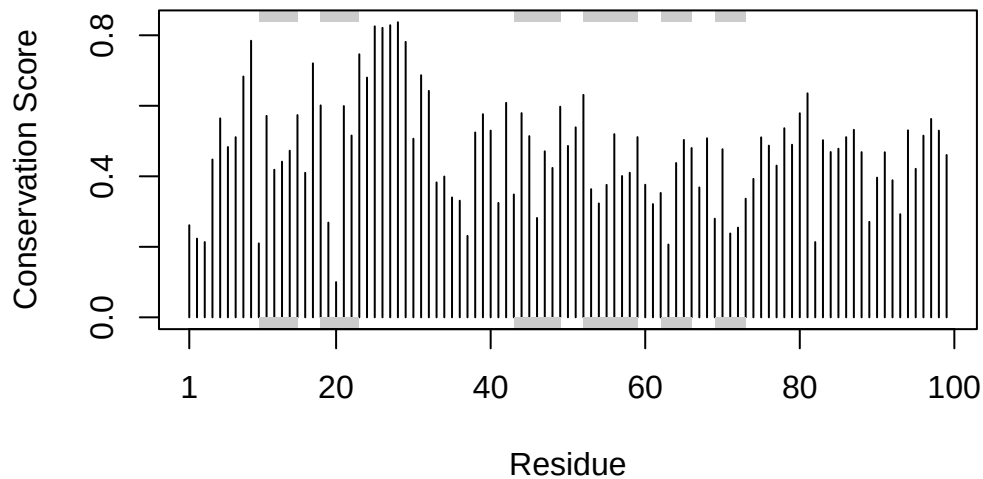
How many sequences are in this alignment

```
dim(aln$ali)
```

```
[1] 5397 132
```

We can score residue conservation in the alignment with the `conserv()` function

```
sim <- conserv(aln)
plotb3(sim[1:99], sse=trim.pdb(pdb, chain="A"),
       ylab="Conservation Score")
```



Note the conserved Active Site residues D25, T26, G27, A28. These positions will stand out if we generate a consensus sequence with a high cutoff value:

```
con <- consensus(aln, cutoff = 0.9)
con$seq
```

```
[1] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[19] "-" "-" "-" "-" "-" "-" "D" "T" "G" "A" "-" "-" "-" "-" "-" "-" "-" "-"
[37] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[55] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[73] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[91] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[109] "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-" "-"
[127] "-" "-" "-" "-" "-" "-"
```

For a final visualization of these functionally important sites we can map this conservation score to the Occupancy column of a PDB file for viewing in molecular viewer programs such as Mol*, PyMol, VMD, chimera etc.

```
m1.pdb <- read.pdb(pdb_files[1])  
occ <- vec2resno(c(sim[1:99], sim[1:99]), m1.pdb$atom$resno)  
write.pdb(m1.pdb, o=occ, file="m1_conserv.pdb")
```

Figure 20

```
img3 <- load.image("Figure 20.png")  
plot(img3)
```

